

機械学習特論A（7回）

佐野夏樹

回帰分析：最小二乗法による解析解

★単回帰モデル

残差

$$S_e = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n \left(y_i - (\hat{\beta}_0 + \hat{\beta}_1 x_i) \right)^2$$

残差平方和

実測値 予測値

これが最小になるような回帰母数の推定値を求める!!

⇒ S_e を β_0, β_1 で偏微分して=0と置く

$$\hat{\beta}_1 = \frac{S_{xy}}{S_{xx}} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

最急降下法による回帰係数の計算

$$S_e(\hat{\beta}_0, \hat{\beta}_1) = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - (\hat{\beta}_0 + \hat{\beta}_1 x_i))^2$$

$$\begin{aligned} \frac{\partial S_e}{\partial \hat{\beta}_0} &= \sum_{i=1}^n \frac{\partial S_e}{\partial e_i} \frac{\partial e_i}{\partial \hat{\beta}_0} \\ &= -2 \sum_{i=1}^n (y_i - (\hat{\beta}_0 + \hat{\beta}_1 x_i)) \end{aligned}$$

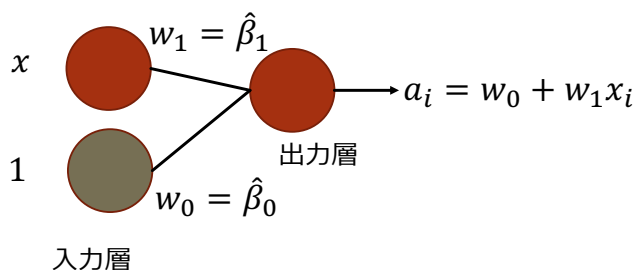
$$\begin{aligned} \frac{\partial S_e}{\partial \hat{\beta}_1} &= \sum_{i=1}^n \frac{\partial S_e}{\partial e_i} \frac{\partial e_i}{\partial \hat{\beta}_1} \\ &= -2 \sum_{i=1}^n (y_i - (\hat{\beta}_0 + \hat{\beta}_1 x_i)) x_i \end{aligned}$$

最急降下法の更新則

$$\hat{\beta}_0 \leftarrow \hat{\beta}_0 - \alpha \frac{\partial S_e}{\partial \hat{\beta}_0}$$

$$\hat{\beta}_1 \leftarrow \hat{\beta}_1 - \alpha \frac{\partial S_e}{\partial \hat{\beta}_1}$$

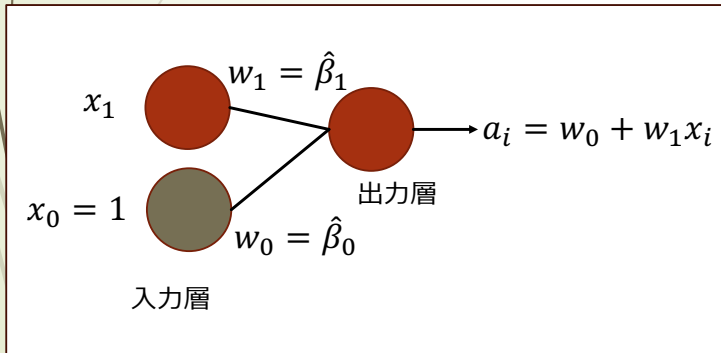
回帰モデルのニューラルネットワーク表現



- 回帰モデルは、2層ニューラルネットワークとして表現できる。ニューラルネットワークは通常は、多層（深層）であり、中間層の素子において活性化関数による変換が行われる。
- 入力層の緑色の素子は、バイアス素子と呼ばれ、常に1が入力される。
- $\hat{y}_i = a_i = w_0 \times x_0 + w_1 \times x_1$ の計算結果が出力層の素子に入力され、出力層の素子では、何も変換せず、出力する（線形出力）。
- ニューラルネットワークにおいて、素子と素子をつなぐ重み(w_0, w_1)は、回帰モデルの回帰係数($\hat{\beta}_0, \hat{\beta}_1$)に相当する。 a_i は、回帰式の予測値 $\hat{y}_i$ に対応する。
- ニューラルネットワークにおける重み w の更新則である誤差逆伝播は、基本的に最急降下法をやっている。

最小二乗誤差と δ 表記

$$E(w_0, w_1) = \frac{1}{2} \sum_{i=1}^n (a_i - y_i)^2 = \frac{1}{2} \sum_{i=1}^n ((w_0 + w_1 x_i) - y_i)^2$$



- 出力層の出力 a_i と教師変数 y_i の間の2乗誤差である E を最小化する.
- E に係数 $\frac{1}{2}$ がついているのは, $\frac{\partial E}{\partial w_0}$, $\frac{\partial E}{\partial w_1}$ を計算した時に係数を1にして式が煩雑にならない様にするため.

表記 δ_i の導入

$$\delta_i = \frac{\partial E}{\partial a_i} = a_i - y_i$$

最急降下法による回帰係数の計算 (δ 表記)

$$E(w_0, w_1) = \frac{1}{2} \sum_{i=1}^n (\underbrace{(w_0 + w_1 x_i)}_{a_i} - y_i)^2$$

$$\delta_i = \frac{\partial E}{\partial a_i} = a_i - y_i$$

$$\frac{\partial a_i}{\partial w_0} = 1, \frac{\partial a_i}{\partial w_1} = x_i$$

$$\begin{aligned} \frac{\partial E}{\partial w_0} &= \sum_{i=1}^n \frac{\partial E}{\partial a_i} \frac{\partial a_i}{\partial w_0} \\ &= \sum_{i=1}^n \delta_i \end{aligned}$$

$$\begin{aligned} \frac{\partial E}{\partial w_1} &= \sum_{i=1}^n \frac{\partial E}{\partial a_i} \frac{\partial a_i}{\partial w_1} \\ &= \sum_{i=1}^n \delta_i x_i \end{aligned}$$

最急降下法の更新則

$$w_0 \leftarrow w_0 - \alpha \frac{\partial E}{\partial w_0}$$

$$w_1 \leftarrow w_1 - \alpha \frac{\partial E}{\partial w_1}$$

δ表記による勾配計算のイメージ

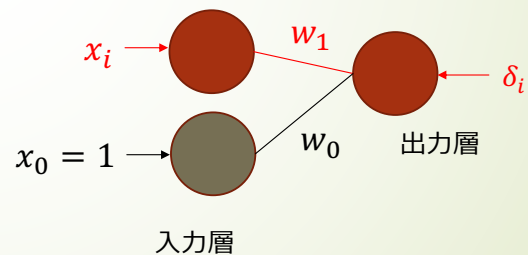
$$E(w_0, w_1) = \frac{1}{2} \sum_{i=1}^n \underbrace{((w_0 + w_1 x_i) - y_i)^2}_{\delta_i}$$

$$\frac{\partial E}{\partial w_0} = \sum_{i=1}^n \delta_i$$

$$\frac{\partial E}{\partial w_1} = \sum_{i=1}^n \delta_i x_i$$

$\frac{\partial E}{\partial w_1}$ の計算は、出力層からの δ_i に
入力層の x_i をかける。

$\frac{\partial E}{\partial w_0}$ の計算は、出力層からの δ_i に
入力層の $x_0 = 1$ をかける。



最急降下法による回帰係数の計算手続き

1. 閾値 $\varepsilon > 0$, ステップ幅 $\alpha > 0$ を設定する
2. w_0, w_1 の初期値 w_0^0, w_1^0 を設定し, $t=0$ から, 以下3,4,5を終了条件を繰り返す.
3. $\frac{\partial E}{\partial w_0}(w_0^t, w_1^t) = \sum_{i=1}^n \delta_i^t, \frac{\partial E}{\partial w_1}(w_0^t, w_1^t) = \sum_{i=1}^n \delta_i^t x_i$ の値を計算する.
4. w_0, w_1 の値を更新する. w_0^t, w_1^t, δ_i^t は t 回目の更新後の w_0, w_1, δ_i の値

$$w_0^{t+1} = w_0^t - \alpha \frac{\partial E}{\partial w_0}(w_0^t, w_1^t)$$

$$w_1^{t+1} = w_1^t - \alpha \frac{\partial E}{\partial w_1}(w_0^t, w_1^t)$$
5. 終了判定
 $\left| \frac{\partial E}{\partial w_0}(w_0^t, w_1^t) \right| + \left| \frac{\partial E}{\partial w_1}(w_0^t, w_1^t) \right| < \varepsilon$ ならば終了 そうでなければ, 3に戻る

この方法は, w_0, w_1 の値の更新に全データを用いているので, **バッチ学習**と呼ばれる.
一方, 1サンプル毎に w_0, w_1 の値を更新する方法を **オンライン学習** と呼ぶ.

Pythonを用いたirisデータへの適用（1）

アヤメ（iris）のデータのsepal length（ガクの長さ）をx, petal length（花びらの長さ）をyとして、xからyを予測する回帰モデルの回帰係数を最急降下法を用いて計算する。

```
#sklearnのデータセットからアヤメのデータをインポートする
from sklearn.datasets import load_iris
import pandas as pd
import numpy as np
iris = load_iris()
#pandasのDataFrameを作成。columnsで列見出しを指定
iris_df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
print(iris_df)
```

Pythonを用いたirisデータへの適用（2）

各パラメータの値とwの初期値設定, データの準備を行う。

```
import numpy as np
#Iter:繰り返し数上限, EPS:収束判定パラメータ, alpha:ステップ幅のパラメータ
Iter=100000;EPS=0.01;alpha=0.0001
#重み(回帰係数)wの初期値を設定
w=np.array([0,0])
#サンプル数(iris_dfの行数)を取得し, nに入れる
n=len(iris_df)
#iris_dfの0列目(sepal length)をsepに入れる
sep=iris_df.iloc[:,1]
#1をn個縦に並べたベクトルとxをhstack関数で連結し, 改めてxとする。
x=np.hstack([np.ones((n,1)),sep])
#iris_dfの2列目(petal length)をyに入れる
y=iris_df.iloc[:,2]
```

Pythonを用いたirisデータへの適用（3）

収束するまでwの更新を行う。

```
for i in range(Iter+1):
    # 予測値aを計算(dot関数は行列積を計算)
    a=np.dot(x,w)
    # 誤差deltaを計算
    delta=a-y
    # deltaを転置した後、xとの行列積を計算し、grad(1番目の要素はdeltaの和、2番目の要素はdeltaとxの積和)とする。
    grad=np.dot(np.transpose(delta),x)
    # 最急降下法によるwの更新
    w=w-alpha*grad
    # deltaを転置した後、deltaと行列積を計算することで誤差二乗和sseを計算する。
    sse=np.dot(np.transpose(delta),delta)
    # 更新情報を逐次確認する場合は、次のコメントアウトを外す
    print("i={},w0={},w1={},sse={}".format(i,w[0],w[1],sse))
    # 収束判定で使用する勾配gradの絶対和をsagとする。absは絶対値を計算する関数
    sag=sum(abs(grad))
    # sagがEPSよりも小さかったら、"Converged"と表示し、for文から抜ける。
    if sag<EPS:
        print("Converged")
        break
    # 最終結果を表示する。
print("i={},w0={},w1={},sse={}".format(i,w[0],w[1],sse))
```

Pythonを用いたirisデータへの適用（4）

```
from sklearn.linear_model import LinearRegression
model_lr=LinearRegression()
model_lr.fit(sep,y)
```

```
LinearRegression()
LinearRegression()
```

```
print(model_lr.intercept_,model_lr.coef_)
-7.101443369602455 [1.85843298]
```

sklearnパッケージのLinearRegression関数を用いて回帰係数を計算してみる。LinearRegression関数では、最急降下法を使って求めているわけではないが、およそ近い値になっていることがわかる。

Pythonを用いたirisデータへの適用（5）

sepal length（ガクの長さ）に加えて, sepal width（ガクの幅）を説明変数に加えてxとして, y, petal length（花びらの長さ）を予測する重回帰モデルの回帰係数を最急降下法を用いて計算する.

wの要素を一つ追加し,
0を加える.

Sepal widthも加える

```
import numpy as np
#Iter:繰り返し数上限, EPS:収束判定パラメータ, alpha:ステップ幅のパラメータ
Iter=100000;EPS=0.01;alpha=0.0001
#重み(回帰係数)wの初期値を設定
w=np.array([0,0,0])
#サンプル数(iris_dfの行数)を取得し, nに入れる
n=len(iris_df)
#iris_dfの0列目(sepal length)と1列目(sepal width)をsepに入れる
sep=iris_df.iloc[:,2]
#1をn個縦に並べたベクトルとxをhstack関数で連結し, 改めてxとする.
x=np.hstack([np.ones((n,1)),sep])
#iris_dfの2列目(petal length)をyに入れる
y=iris_df.iloc[:,2]
```

Pythonを用いたirisデータへの適用（6）

```
for i in range(Iter+1):
    #予測値aを計算(dot関数は行列積を計算)
    a=np.dot(x,w)
    #誤差deltaを計算
    delta=a-y
    #deltaを転置した後, xとの行列積を計算し, grad(1番目の要素はdeltaの和, 2番目の要素はdeltaとxの積和)とする.
    grad=np.dot(np.transpose(delta),x)
    #最急降下法によるwの更新
    w=w-alpha*grad
    #deltaを転置した後, deltaと行列積を計算することで誤差二乗和sseを計算する.
    sse=np.dot(np.transpose(delta),delta)
    #更新情報を逐次確認する場合は, 次のコメントアウトを外す
    print("i={},w0={},w1={},w2={},sse={}".format(i,w[0],w[1],w[2],sse))
    #収束判定で使用する勾配gradの絶対和をsagとする. absは絶対値を計算する関数
    sag=sum(abs(grad))
    #sagがEPSよりも小さかったら, "Converged"と表示し, for文から抜ける.
    if sag<EPS:
        print("Converged")
        break
    #最終結果を表示する.
    print("i={},w0={},w1={},w2={},sse={}".format(i,w[0],w[1],w[2],sse))
```

w2の計算結果も出力する.

Pythonを用いたirisデータへの適用（7）

```
from sklearn.linear_model import LinearRegression
model_lr=LinearRegression()
model_lr.fit(sep,y)
```

▼ LinearRegression

LinearRegression()

```
print(model_lr.intercept_,model_lr.coef_)
```

```
-2.524761511833407 [ 1.77559255 -1.33862329]
```